

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO4: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 30%

Dr G A. SENTHIL

QUESTIONS: 10 Total Marks: 50

Annexure A

Mock Interview

Duration: 1hr

The following are the answers to five of the ten mock-interview questions, presented in the same order as the original question paper. Each response is written as it would be delivered in an actual technical interview: a clear explanation of the underlying concept, followed by how it is applied in the given real-world scenario.

Question 1 (5 Marks)

A machine learning engineer applies Reinforcement Learning to solve sequential decision-making problems. Explain how an agent learns optimal behavior through interaction with the environment.

Reinforcement Learning frames a problem as a continuous loop of interaction between an agent and its environment, formalised as a Markov Decision Process (MDP) with states (S), actions (A), a transition function $P(s'|s,a)$, and a reward function $R(s,a)$. At every timestep the agent observes the current state, selects an action according to its policy $\pi(a|s)$, and receives a reward together with the next state from the environment. There is no labelled training data as in supervised learning — instead, the agent must discover which actions are good purely through trial and error and the reward signal it receives.

Learning optimal behaviour means the agent gradually improves its policy so as to maximise the expected cumulative discounted reward, $G_t = \sum \gamma^k r_{t+k+1}$, rather than just the immediate reward. This is achieved through value-based methods (e.g. Q-learning, where the agent learns an action-value function $Q(s,a)$ estimating long-term return and updates it using the Bellman equation), policy-based methods (where the policy itself is parameterised and updated via gradient ascent on expected reward, e.g. REINFORCE, PPO), or actor-critic hybrids that combine both.

Two forces balance this learning process: exploration, where the agent tries new or uncertain actions to discover potentially better strategies (e.g. epsilon-greedy or entropy bonuses), and exploitation, where it uses its current best-known policy to maximise reward. Over many episodes, as the agent's value estimates or policy parameters converge, its behaviour shifts from largely random exploration toward a near-optimal, exploitation-driven policy that reliably maximises long-term reward — this is the essence of how an RL agent 'learns' purely from interaction, without being explicitly told the correct action at each step.

Question 2 (5 Marks)

If you are asked to design a Reinforcement Learning solution for a real-time system such as ride-sharing or delivery optimization, explain how you would formulate the problem using a Markov Decision Process.

The first step is to define the five components of the MDP tuple (S, A, P, R, γ) for the specific business problem:

- State (S): a snapshot of the system relevant to the decision — e.g. driver/vehicle locations, current demand heatmap, number of pending ride/delivery requests, time of day, traffic conditions, and each vehicle's remaining capacity or battery/fuel level.
- Action (A): the decision the agent can make at each timestep — e.g. which driver to assign to which request, whether to reposition an idle vehicle, or which delivery route/order sequence to choose next.
- Transition function (P): how the state evolves after an action — new rider requests arriving stochastically, traffic conditions changing, vehicles moving toward their destinations — this is typically not known explicitly and is instead sampled from a simulator or historical/live data (making it a natural fit for model-free RL).
- Reward (R): a signal engineered to reflect the real business objective — e.g. positive reward for completed trips/deliveries weighted by revenue, negative reward for rider/customer wait time, cancellations, or fuel/distance cost, and possibly a small penalty per timestep to encourage efficiency.
- Discount factor (γ): chosen close to 1 (e.g. 0.95–0.99) since ride-sharing/delivery decisions have consequences that unfold over an entire shift, not just the next few seconds.

Because the environment is real-time and highly dynamic, the practical formulation typically treats this as a semi-MDP or multi-agent MDP (one agent per vehicle, or a single centralised dispatcher agent), trained first in an offline simulator built from historical trip data before being deployed with careful monitoring, since online exploration directly on live customers carries real financial and safety risk.

Question 3 (5 Marks)

Compare model-free and model-based Reinforcement Learning approaches. Explain their working principles and how they handle uncertainty.

Model-free RL learns a policy or value function directly from experience, without ever explicitly learning the environment's transition function $P(s'|s,a)$ or reward function. Algorithms such as Q-learning, SARSA, DQN, and policy-gradient methods (REINFORCE, PPO, A2C) fall into this category. The agent essentially treats the environment as a black box: it repeatedly samples (state, action, reward, next-state) transitions and uses them to improve its estimate of value or its policy, converging asymptotically to the optimal behaviour given enough experience.

Model-based RL, in contrast, explicitly learns (or is given) a model of the environment's dynamics — an approximation of $P(s'|s,a)$ and $R(s,a)$. The agent can then use this model to simulate future trajectories internally and plan ahead, for example via Monte-Carlo Tree Search (as in AlphaZero) or trajectory optimisation (as in the Dyna architecture or MuZero), rather than relying solely on real interaction.

The key trade-off is sample efficiency versus computational cost and model bias. Model-based methods are typically far more sample-efficient because every learned model update can be 'replayed' through imagined rollouts, which is valuable when real-world interaction is expensive or risky (e.g. robotics, healthcare). However, their performance is capped by how accurate the learned model is — if the model is wrong, planning through it compounds those errors. Model-free methods avoid this model-bias problem entirely since they learn directly from ground-truth experience, but typically require many more real interactions to converge.

Regarding uncertainty: model-free methods handle it implicitly through the exploration strategy (epsilon-greedy, upper-confidence bounds, or entropy regularisation) and through averaging over many samples, without ever representing uncertainty explicitly. Model-based methods can represent uncertainty more explicitly — for instance, an ensemble of learned dynamics models (as in PETS) or a Bayesian dynamics model can quantify how confident the agent is about a predicted transition, and the agent can then act cautiously (or seek information) in high-uncertainty regions of the state space, giving model-based RL a natural mechanism for risk-aware or safe exploration that model-free methods lack by default.

Question 4 (5 Marks)

A recommendation system using Reinforcement Learning is underperforming. Explain how reward sparsity and hyperparameter tuning can be diagnosed and improved.

Reward sparsity is a very common failure mode in RL-based recommenders: if the agent is only rewarded when a user makes a purchase or long-dwell click, and such events are rare relative to the huge action space of possible items, the agent receives almost no learning signal and its value estimates stay close to their initial (often zero) values for most state-action pairs. Diagnosing this typically starts by logging the reward distribution and the fraction of timesteps with non-zero reward across training — if that fraction is extremely low, sparsity is likely the primary bottleneck, which can be confirmed by checking whether the agent's policy stays close to its initial random/uniform distribution over many episodes.

Common fixes for reward sparsity include reward shaping (adding smaller intermediate rewards for proxy engagement signals such as clicks, watch-time, or add-to-cart, carefully weighted so they do not distort the true long-term objective), curriculum learning (starting with an easier or denser reward signal and gradually shifting to the true sparse objective), and using

auxiliary/multi-task objectives (e.g. predicting whether the user will engage at all as an additional learning signal) to keep gradients informative even in the absence of the main reward.

Separately, hyperparameter issues can compound a sparsity problem or independently cause underperformance. Diagnosis follows a systematic sweep: the learning rate is checked first (a rate too high causes oscillating/diverging value estimates; too low causes painfully slow convergence, visible in a flat or noisy loss curve), followed by the discount factor γ (too low ignores long-term user retention effects; too high makes credit assignment harder), the exploration rate/schedule (insufficient exploration means the agent never discovers better items to recommend; too much exploration hurts short-term user experience), and the replay buffer size/batch size for off-policy methods. In practice these are tuned via a structured search (grid search, Bayesian optimisation, or population-based training) evaluated on a held-out offline simulator or through careful, guarded online A/B tests, rather than by manual trial-and-error alone.

Question 5 (5 Marks)

If you are implementing an RL-based game-playing AI, describe how you would design the learning pipeline and ensure training stability.

The pipeline begins with the environment interface: wrapping the game so it exposes a standard `step()/reset()` API returning observations (raw pixels, game state vectors, or a symbolic board representation), rewards, and episode-termination signals — typically built on top of a framework such as Gymnasium/OpenAI Gym so that any standard RL library (Stable-Baselines3, RLlib) can be plugged in directly.

- Algorithm selection: value-based methods such as DQN (with target networks and experience replay) for discrete-action games, or policy-gradient/actor-critic methods such as PPO or A3C for continuous or very large action spaces; self-play is added for competitive two-player games.
- Network architecture: convolutional layers for pixel-based games, or fully-connected/transformer layers for structured state vectors, sized appropriately to avoid over- or under-fitting the state representation.
- Reward design: usually the game's own score, sometimes supplemented with shaping rewards for intermediate progress if the native reward is too sparse.
- Training infrastructure: parallelised environment workers (vectorised environments) to gather experience faster, and a replay buffer or on-policy rollout buffer depending on the chosen algorithm.

Training stability is then ensured through several standard mechanisms: (1) a target network updated slowly (or via Polyak/soft updates) to prevent the moving-target problem in value-based methods; (2) gradient clipping to stop occasional large updates from destabilising the network; (3) reward and/or observation normalisation so magnitudes stay in a numerically well-behaved range; (4) an appropriately small, often decayed learning rate with a stable optimiser such as Adam; (5) trust-region or clipped-objective techniques (as in PPO) to prevent any single policy update from moving too far from the previous policy; and (6) frequent evaluation checkpoints against a fixed benchmark (e.g. win-rate against a scripted or frozen previous version of the agent) so that regressions or divergence are caught early rather than discovered only at the end of a long training run.

Code of Conduct:

I A SHIVA SHANKAR REDDY (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A SHIVA SHANKAR REDDY

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately